

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naive Bayes and KNN
Student Name	S.S.Janane
Register Number	3122247001024
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	27/07/26

1 Objective

This experiment builds and compares two families of spam classifiers on the Spambase email dataset:

Build spam/ham classifiers using Naïve Bayes and K-Nearest Neighbors (KNN) and to compare the Gaussian, Multinomial, and Bernoulli variants of Naive Bayes against each other. To study how classifier performance changes as the neighborhood size k is varied in KNN. Compare KDTree and BallTree as neighbor-search structures for KNN. Tune KNN's hyperparameters using both GridSearchCV and RandomizedSearchCV, and compare the two search strategies. Measure and compare the theoretical and actual (wall-clock) computational cost of every model. Validate the final models using 5-fold cross-validation rather than relying on a single train/test split.

2 Problem Statement

The Spambase dataset contains emails represented by 57 numerical features, including word frequencies, character frequencies, and statistics about capital-letter usage. The objective is to classify each email as spam (1) or ham (0). This experiment compares two different classification approaches—Naïve Bayes, which is probability-based, and K-Nearest Neighbors (KNN), which is distance-based—to evaluate their accuracy, efficiency, and overall suitability for spam detection.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository (mirrored on Kaggle: https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601 raw rows; 4210 duplicate rows
Number of Features	57 numeric features (word-frequency, character-frequency, and capital-run-length statistics)
Number of Classes	2 ; spam (1) and ham (0)
Missing Values	None found in any of the 58 columns
Train-Test Split	Stratified 70/10/20 split into train (2,946), validation (421), and test (843) sets

4 Exploratory Data Analysis

Before touching any model, the raw dataset was profiled to understand its shape, its class balance, and how well-behaved (or not) its 57 features actually are.

- Dataset overview and statistical summary (shape, dtypes, `describe()`)
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution (histograms and boxplots)

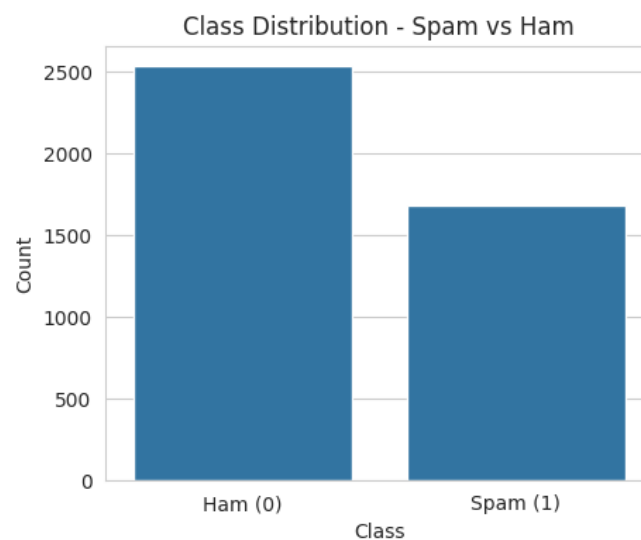


Figure 1: Class distribution of the Spambase dataset after de-duplication

Inference: The dataset has more ham emails than spam emails, but the difference is not very large. A stratified train-test split was used to maintain this balance, and evaluation focuses on metrics like Precision, Recall, F1-score, and ROC-AUC instead of accuracy alone.

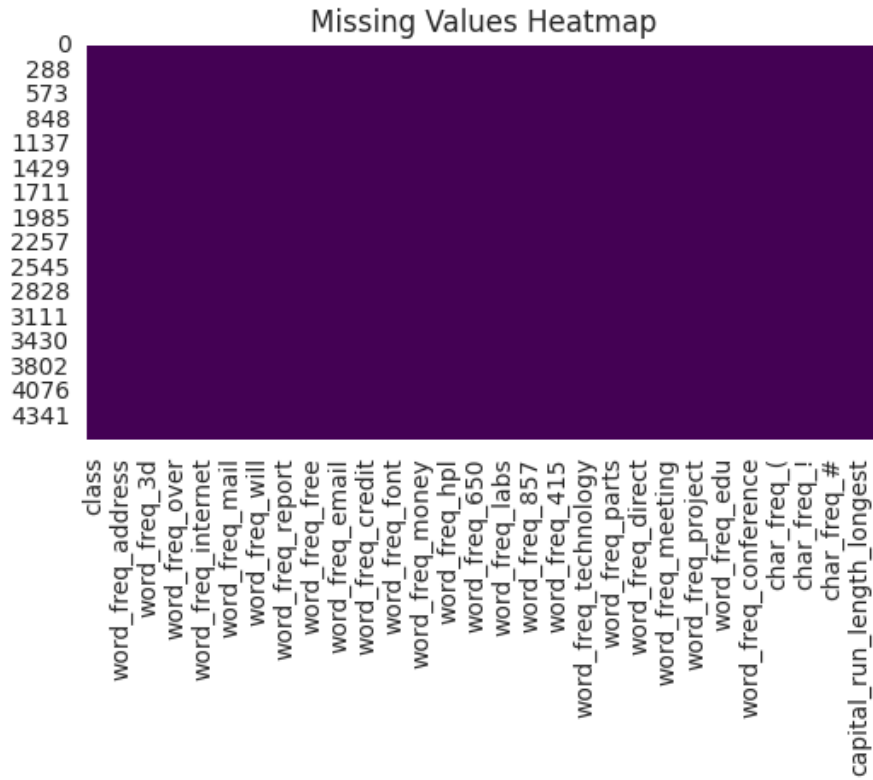


Figure 2: Missing value heatmap across all 58 columns

Inference: There are no missing values in the dataset. Since all features are complete, no data imputation was required before training the models.

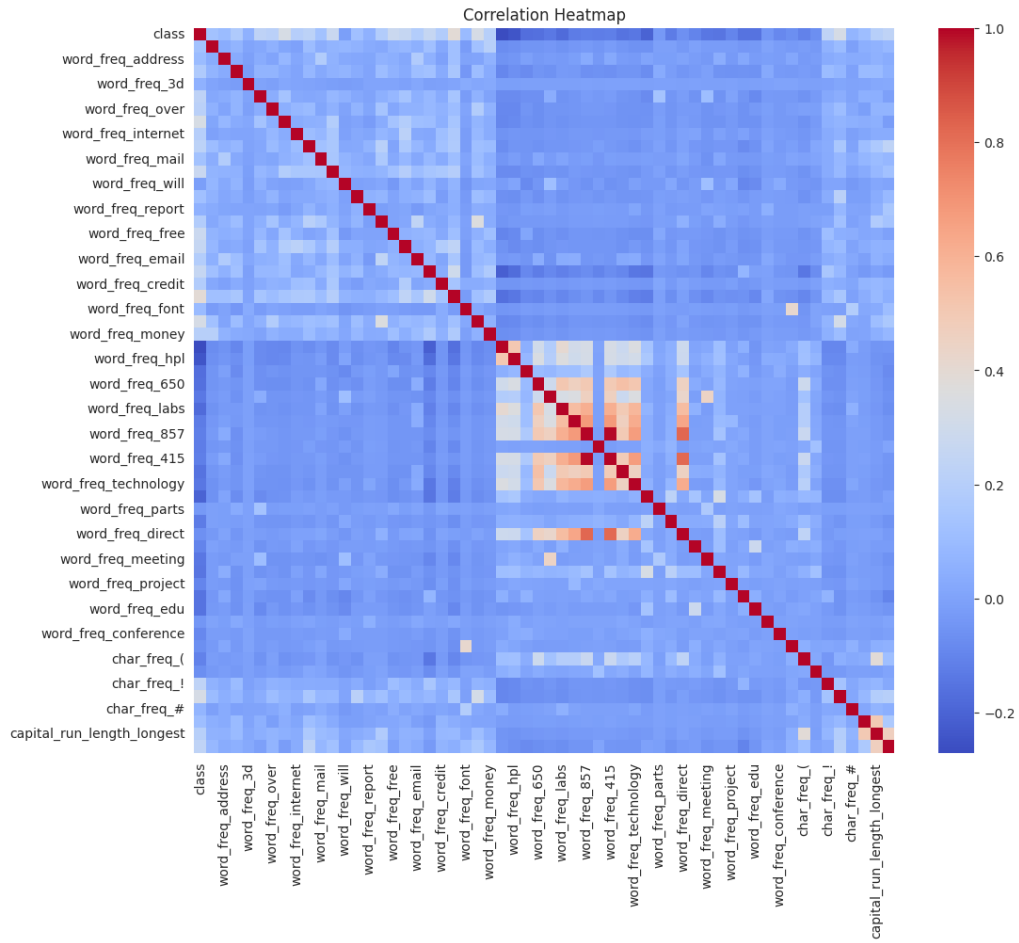


Figure 3: Correlation heatmap of the 57 numeric features

Inference: Most features have low correlation with each other, meaning they provide different information. The capital-letter features are more strongly correlated because they measure similar characteristics of an email.

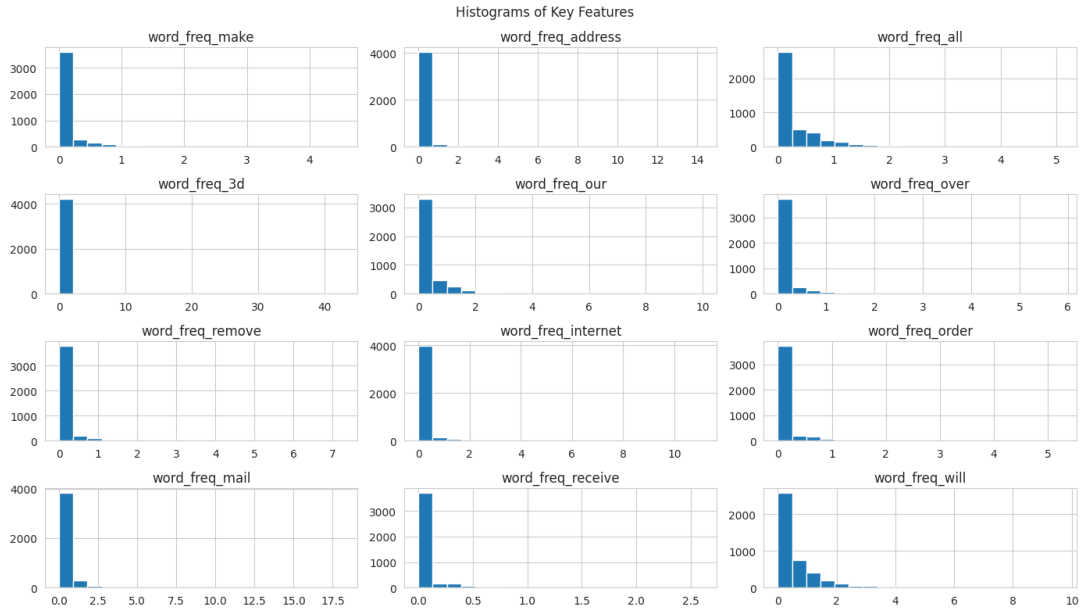


Figure 4: Histograms of the first twelve numeric features

Inference: Most features are heavily skewed, with many emails having zero values for specific words or characters. This shows that the data is not normally distributed, which may affect Gaussian Naïve Bayes.

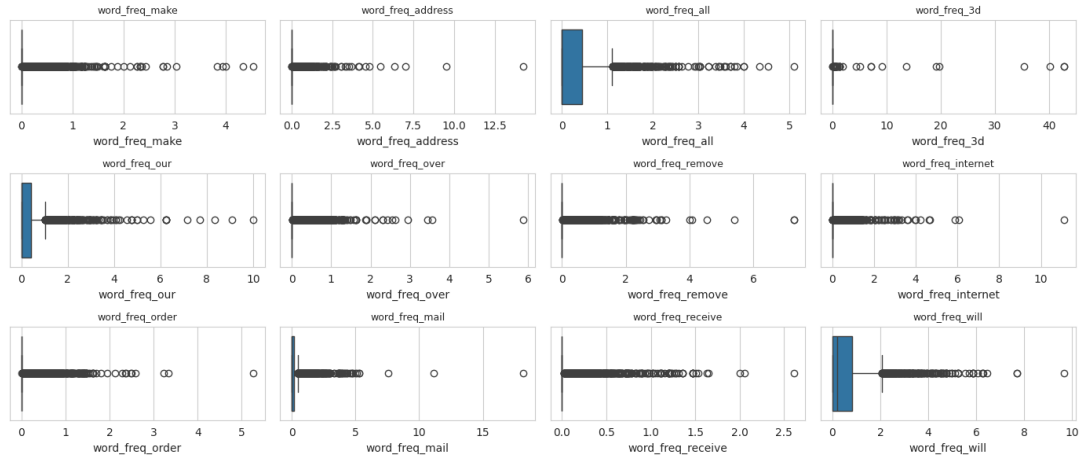


Figure 5: Boxplots of the same twelve numeric features

Inference: Several features contain outliers, especially the capital-letter statistics. These outliers were kept because they are useful indicators of spam rather than errors in the data.

5 Data Preprocessing

- **Duplicate removal:** 391 exact duplicate rows (8.5% of the raw 4,601 rows) were dropped, leaving 4,210 unique rows.
- **Missing value handling:** none were present, but the pipeline still runs a median-fill for numeric columns and a mode-fill for categorical columns as a safety net.

- **Encoding:** not required. All 57 Spambase features are already numeric (word/character frequencies and capital-run-length statistics), and the target column is already a clean 0/1 integer.
- **Constant-column check:** none of the 57 features had zero variance, so nothing was dropped at this stage.
- **Feature scaling:** three separately-scaled copies of the feature matrix were prepared, because each Naïve Bayes variant expects a different kind of input:
 - **StandardScaler** (zero mean, unit variance) → Gaussian NB and every KNN model, since KNN’s distance calculation is directly sensitive to feature scale.
 - **MinMaxScaler** (0–1 range) → Multinomial NB, which requires non-negative input.
 - **Binarizer** (thresholded at 0, i.e. “did this word appear at all”) → Bernoulli NB.
- **Train/validation/test split:** a stratified 70/10/20 split (2,946 / 421 / 843 rows) was produced with two chained calls to `train_test_split`, both using `stratify=y` so that the 60/40 class ratio is preserved in every partition. The validation set drives the k -sweep and the choice of the best Naïve Bayes variant; the test set is only touched once, for the numbers reported in Section 10.

Listing 1: Preparing the three scaled feature sets

```

1 std_scaler = StandardScaler()
2 X_train_std = std_scaler.fit_transform(X_train)
3 X_val_std   = std_scaler.transform(X_val)
4 X_test_std  = std_scaler.transform(X_test)
5
6 mm_scaler = MinMaxScaler()
7 X_train_mm = mm_scaler.fit_transform(X_train)
8
9 binarizer = Binarizer(threshold=0.0)
10 X_train_bin = binarizer.fit_transform(X_train)

```

6 Machine Learning Algorithm

6.1 Naïve Bayes

Naïve Bayes is a probabilistic classifier built directly on Bayes’ theorem:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)} \quad (1)$$

The “naïve” part is the assumption that every feature is conditionally independent given the class, which collapses the otherwise intractable joint likelihood into a simple product:

$$P(X | C) = \prod_{i=1}^d P(x_i | C) \quad (2)$$

The three variants compared in this experiment differ only in how $P(x_i | C)$ is modeled:

- **Gaussian NB** assumes each feature is normally distributed within a class: $P(x_i | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$.
- **Multinomial NB** treats each feature as a count drawn from a multinomial distribution, with additive (Laplace) smoothing to avoid zero probabilities for unseen counts.
- **Bernoulli NB** treats each feature as a binary presence/absence indicator, and – unlike Multinomial NB – explicitly penalizes a word’s *absence*, not just its presence.

Advantages: training is linear in the number of samples and features ($O(nd)$), prediction is essentially instantaneous ($O(d)$), it needs comparatively little training data, and it degrades gracefully in high dimensions. **Limitations:** the conditional-independence assumption is almost never true in practice – words in an email are not independent of one another – and the Gaussian variant in particular is hurt by the heavily right-skewed, non-Gaussian shape of most Spambase features, as seen in Section 4.

6.2 K-Nearest Neighbors (KNN)

KNN is a lazy, instance-based learner: it does not fit a model in the usual sense, it simply stores the training data and defers all computation to prediction time. To classify a new point, it finds the k closest training points under a chosen distance metric and assigns the majority class among them (optionally weighting each neighbor by $1/\text{distance}$ instead of voting uniformly). Two distance metrics were compared in this experiment:

$$\text{Euclidean: } d(x, y) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2} \quad (3)$$

$$\text{Manhattan: } d(x, y) = \sum_{i=1}^d |x_i - y_i| \quad (4)$$

Finding the k nearest neighbors by brute force means comparing a query point against every training point, which is where KDTree and BallTree come in – both organize the training set into a spatial tree so that most points can be discarded without an explicit distance calculation, turning an average-case linear search into a logarithmic one.

Advantages: no distributional assumption about the data, naturally captures non-linear decision boundaries, and distance-weighting lets closer, more relevant neighbors dominate the vote. **Limitations:** prediction cost grows with the size of the training set (even tree-based search only gives a logarithmic, not constant, improvement), it is sensitive to feature scaling, and with 57 features it is exposed to the curse of dimensionality, where distances between points start to lose their discriminative meaning.

Table 2: Theoretical time complexity (n = training samples, d = features)

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute Force)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ avg.
BallTree	$O(n \log n)$	$O(\log n)$ avg.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

8.1 Naïve Bayes (default parameters)

None of the three Naïve Bayes variants were tuned – they were run with scikit-learn’s default smoothing parameters, which turned out to be a perfectly reasonable baseline given how large the training set is relative to the number of features.

Parameter	Value
<code>var_smoothing</code> (Gaussian NB)	1×10^{-9} (default)
<code>alpha</code> (Multinomial NB)	1.0 (default Laplace smoothing)
<code>alpha</code> (Bernoulli NB)	1.0 (default Laplace smoothing)

8.2 KNN – GridSearchCV vs. RandomizedSearchCV

KNN was tuned over the grid $\{n_neighbors \in [1, 3, 5, 7, 9, 11]\} \times \{weights \in [uniform, distance]\} \times \{algorithm \in [auto, kd_tree, ball_tree]\} \times \{metric \in [euclidean, manhattan]\}$ (72 combinations, 5-fold CV, scored on accuracy). RandomizedSearchCV sampled 20 combinations from the same space, with $n_neighbors$ drawn from a continuous `randint(1, 31)` range.

Table 3: GridSearchCV vs. RandomizedSearchCV – best configuration found by each

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	7
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	auto	ball_tree
CV Accuracy	0.9138	0.9134
Execution Time (s)	20.5732	6.3859

GridSearchCV found a marginally better cross-validated accuracy (0.9138 vs. 0.9134 – a gap of just 0.04 percentage points), but took roughly $3.2\times$ longer to do it, since it exhaustively tries all 72 combinations rather than sampling 20 of them. Both searches independently converged on `manhattan` distance with `distance`-based weighting, which is a good sign that this region of the hyperparameter space is genuinely where the good solutions live, not an artifact of either search strategy. Since GridSearchCV’s score was (very slightly) higher, its configuration – $k = 9$, manhattan distance, distance-weighted voting – was carried forward as the “best KNN” model for the rest of this report.

9 Results

9.1 Naïve Bayes variant comparison (test set)

Table 4: Gaussian vs. Multinomial vs. Bernoulli Naïve Bayes

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8363	0.8932	0.8956
Precision	0.7250	0.9489	0.8827
Recall	0.9494	0.7738	0.8512
F1	0.8222	0.8525	0.8667
ROC-AUC	0.9478	0.9592	0.9562

9.2 Effect of k on KNN (validation set)

Table 5: KNN accuracy, precision, recall, and F1 as k varies

k	Train Acc.	Accuracy	Precision	Recall	F1
1	0.9990	0.9002	0.8889	0.8571	0.8727
3	0.9403	0.9026	0.9097	0.8393	0.8731
5	0.9260	0.9026	0.9097	0.8393	0.8731
7	0.9202	0.8907	0.8861	0.8333	0.8589
9	0.9172	0.8884	0.8954	0.8155	0.8536
11	0.9141	0.8907	0.9013	0.8155	0.8563

9.3 KDTree vs. BallTree (test set, tuned KNN)

Table 6: KDTree vs. BallTree at $k = 9$, manhattan distance, distance weighting

Metric	KDTree	BallTree
Accuracy	0.9181	0.9181
Training Time (s)	0.01611	0.00836
Prediction Time (s)	0.13684	0.08918

9.4 5-Fold Cross-Validation

The best-performing Naïve Bayes variant by validation F1 (Multinomial NB) and the tuned KNN model were both re-validated with 5-fold stratified cross-validation on the training set.

Table 7: 5-fold cross-validation accuracy

Fold	Naïve Bayes (Multinomial)	Best KNN
1	0.9085	0.9153
2	0.8693	0.9117
3	0.8862	0.9253
4	0.8778	0.9066
5	0.8846	0.9049
Average	0.8853	0.9128

9.5 Experimental training and prediction time

Table 8: Measured training and prediction time on the full train/test set

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.009531	0.000600
Multinomial NB	0.002012	0.000196
Bernoulli NB	0.003335	0.000851
Best KNN	0.000843	0.073940

9.6 Final classifier comparison (test set)

Table 9: All four final models, evaluated on the held-out test set

Metric	Best KNN ($k=9$, manhattan, distance-weighted)
Accuracy	0.9181
Precision	0.9406
Recall	0.8482
F1-score	0.8920
ROC-AUC	0.9702

The tuned KNN model beat every Naïve Bayes variant on accuracy, F1, and ROC-AUC (compare against Table in Section 10.1), making it the best single model produced in this experiment.

10 Result Visualization

Every figure below reproduces a plot generated directly from the experiment, with a short inference explaining what it shows.

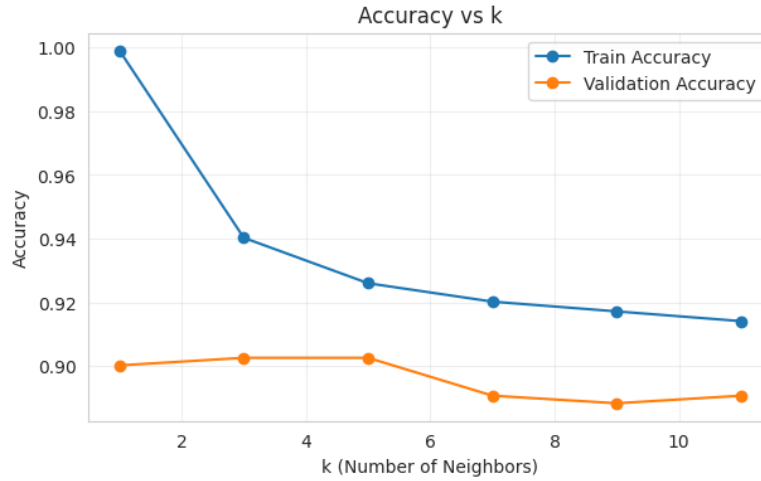


Figure 6: Train vs. validation accuracy as k varies (KNN)

Inference: Training accuracy is highest at $k = 1$, but the validation accuracy is lower, showing overfitting. As k increases, the model becomes more stable, and the best performance is achieved with a moderate value of k .

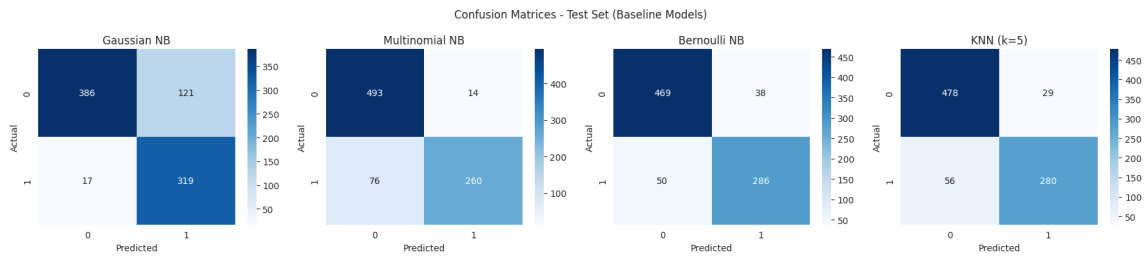


Figure 7: Confusion matrices for all four baseline models (test set)

Inference: Gaussian Naïve Bayes incorrectly classifies more ham emails as spam, while Multinomial Naïve Bayes misses more spam emails. Bernoulli Naïve Bayes and KNN provide a better balance between the two types of errors.

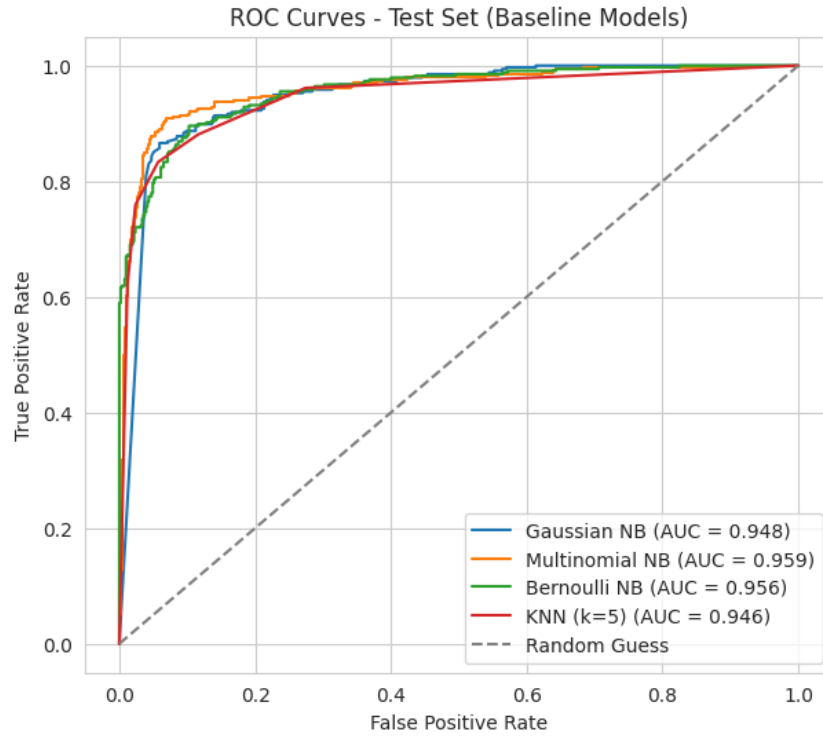


Figure 8: ROC curves for all four baseline models (test set)

Inference: All models perform much better than random guessing. Among the baseline models, Multinomial Naïve Bayes has the highest ROC-AUC, showing better overall classification performance.

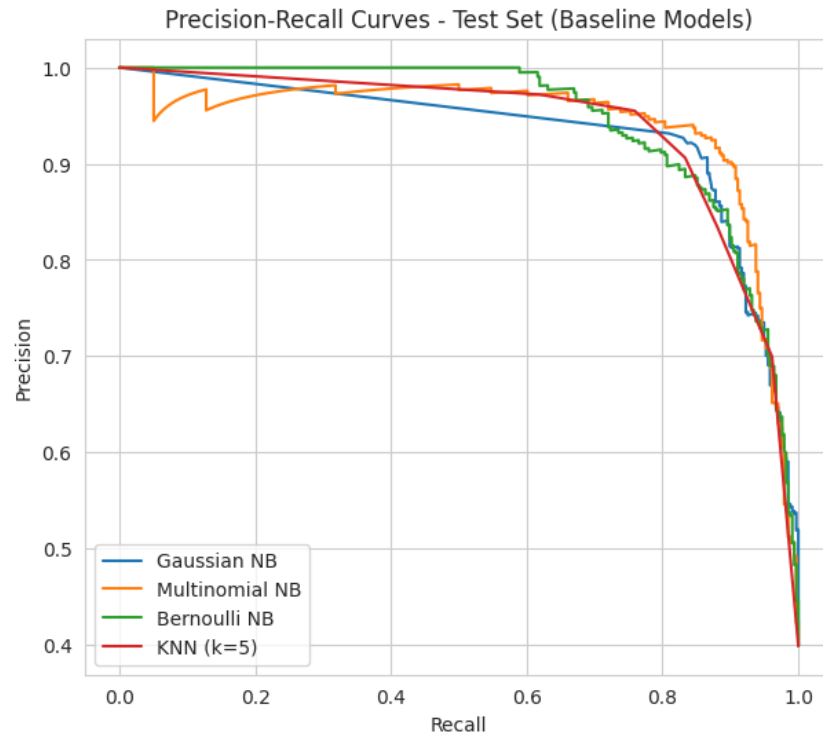


Figure 9: Precision-Recall curves for all four baseline models (test set)

Inference: Multinomial Naïve Bayes and KNN maintain better precision as recall increases. Gaussian Naïve Bayes loses precision more quickly than the other models.

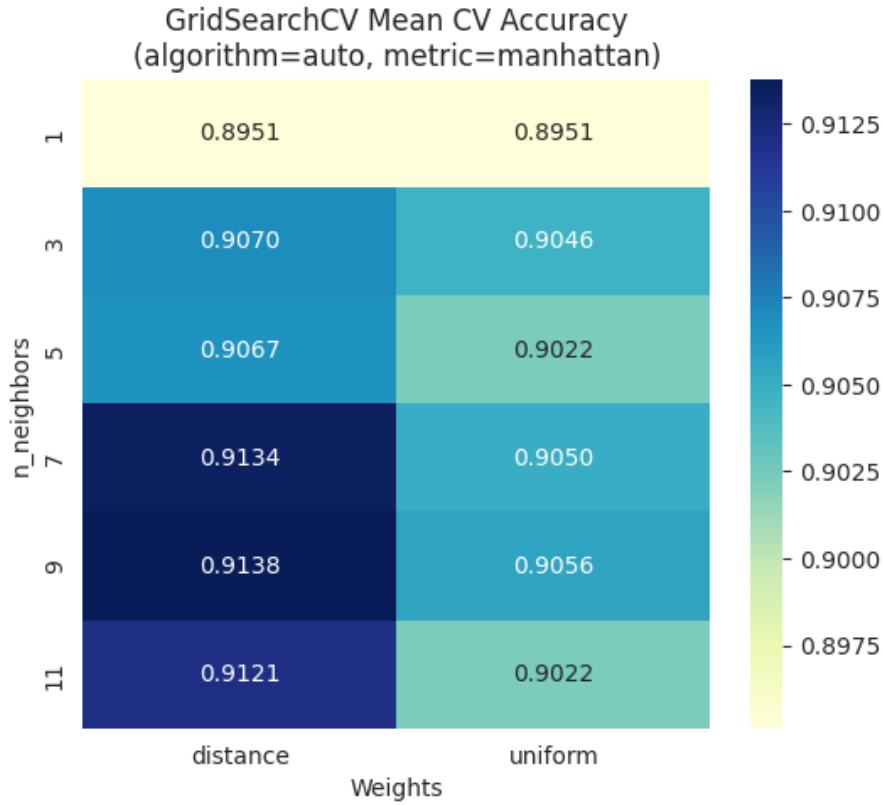


Figure 10: GridSearchCV mean CV accuracy across k and weighting scheme

Inference: With the algorithm and metric fixed at their best-found values, the heatmap makes it clear that distance-weighted voting consistently outperforms uniform voting at every value of k tested, and that accuracy is fairly flat, across $k = 5$ through $k = 11$ once distance weighting is used the search isn't finding one sharp optimum so much as a broad plateau of good configurations.

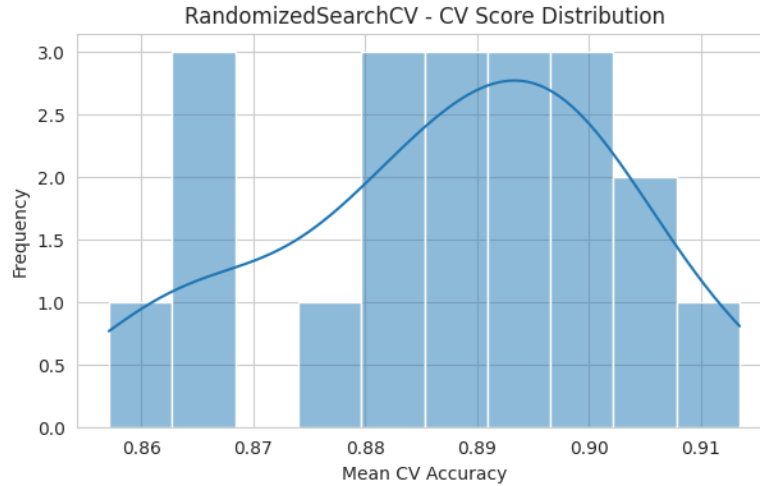


Figure 11: Distribution of mean CV accuracy across RandomizedSearchCV’s 20 sampled configurations

Inference: Most of the 20 randomly sampled configurations cluster in a fairly narrow band of CV accuracy, with only a small number of low-scoring outliers (typically the uniform-weighted, small- k combinations). This is reassuring: it suggests the KNN hyperparameter space for this dataset doesn’t have any hidden traps, and a modest, cheap random search is enough to land close to what an exhaustive grid search finds.

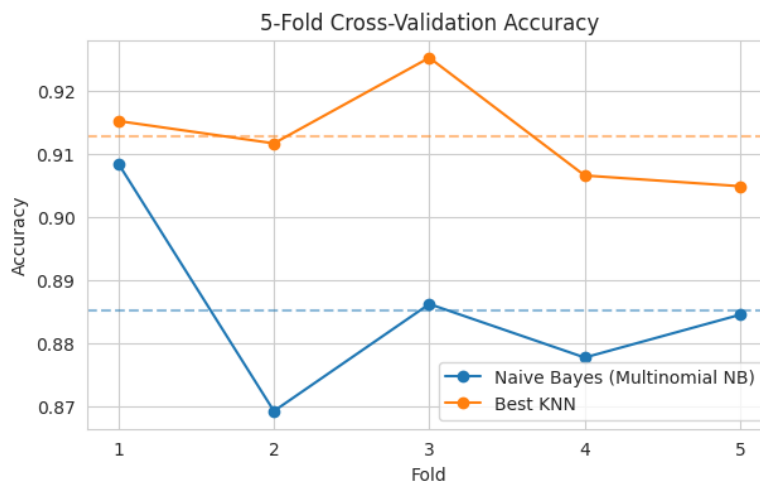


Figure 12: 5-fold cross-validation accuracy, Naïve Bayes vs. KNN

Inference: The tuned KNN model’s fold-to-fold accuracy is both consistently higher than Multinomial NB’s and noticeably less variable (a tighter band around its 0.9128 average than NB’s 0.8853 average), which is a stronger claim than a single train/test split could support on its own it suggests KNN’s advantage here isn’t a lucky split.

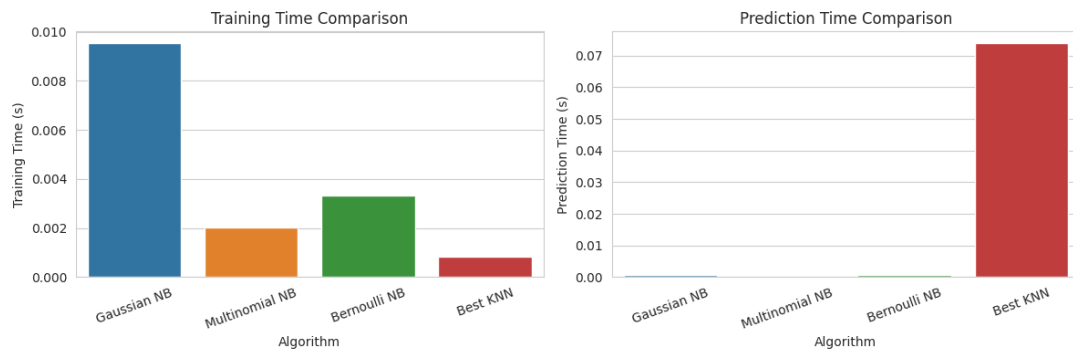


Figure 13: Training time and prediction time comparison across all four models

Inference: The two bar charts are almost mirror images of each other. Best KNN trains almost instantly (0.0008s, since “training” for KNN is just storing the data), but is by far the slowest to predict (0.074s) since every query has to search the tree; the three Naïve Bayes variants show the opposite pattern a small, but non-zero training cost and near-zero prediction cost. This is exactly what the theoretical complexity table in Section 7 predicted.

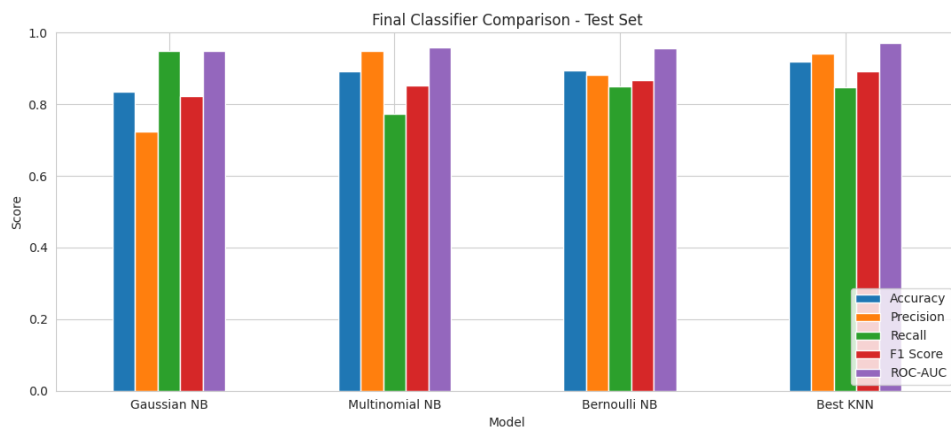


Figure 14: Final classifier comparison bar chart - all metrics, all four models

Inference: The tuned KNN model’s bars are the tallest (or tied for tallest) on every metric except recall, where Gaussian NB’s aggressive spam-catching behaviour still edges it out. Taken together with the ROC-AUC, and F1 columns, this chart is the clearest single piece of evidence in the report, for why tuned KNN was chosen as the best overall model.

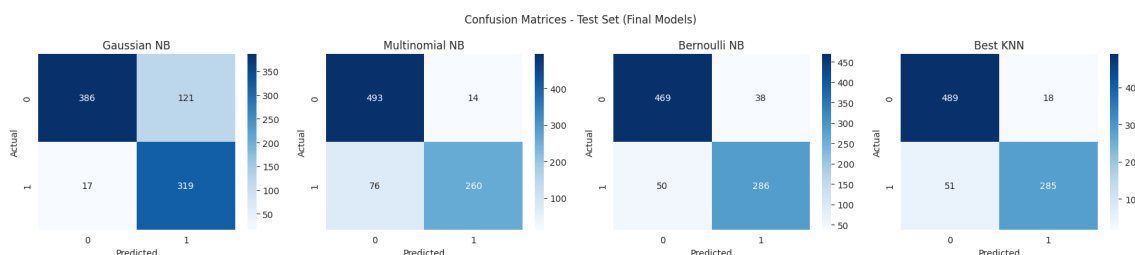


Figure 15: Confusion matrices for the final models (test set)

Inference: Compared to the baseline KNN confusion matrix from Figure 6, tuned KNN cuts false positives from 29 down to 18 while keeping false negatives roughly flat – tangible proof that the GridSearchCV-selected hyperparameters ($k = 9$, manhattan, distance-weighted) genuinely improved the model rather than just improving its cross-validation score in isolation.

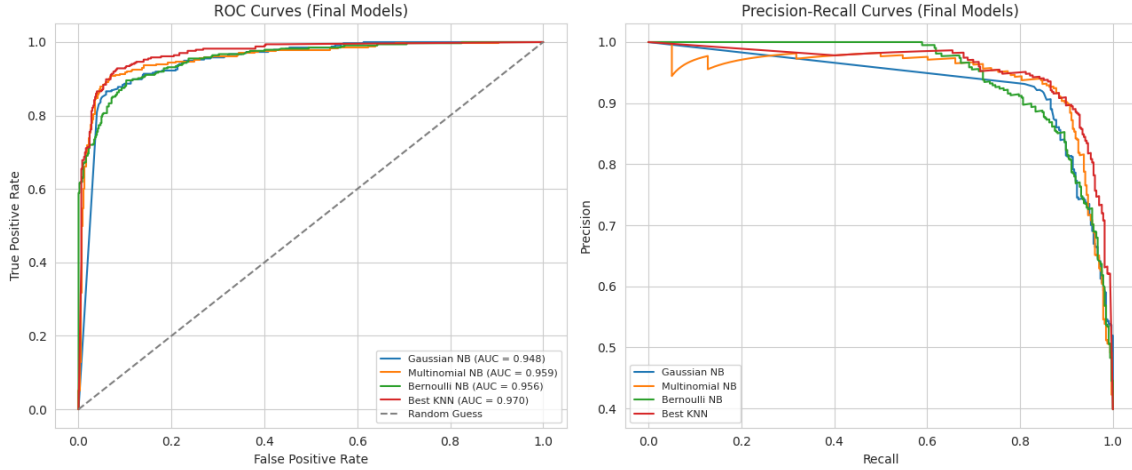


Figure 16: ROC and Precision-Recall curves, final models (test set)

Inference: Tuned KNN’s ROC curve now sits above every Naïve Bayes variant across almost the entire false-positive-rate range, and its ROC-AUC (0.9702) is the highest of the four final models. The precision-recall curve states same: tuned KNN holds a higher precision than the others at any given recall level, confirming that the hyperparameter tuning translated into a real ranking-quality improvement, not just a shift in the default decision threshold.

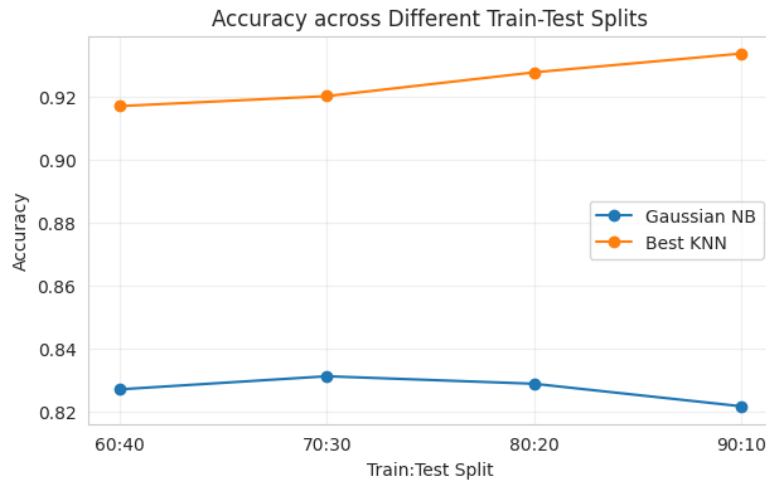


Figure 17: Accuracy of Gaussian NB and Best KNN across different train/test split ratios

Inference: KNN’s accuracy climbs steadily as more data is kept for training (from 91.69% at a 60:40 split up to 93.35% at 90:10), while Gaussian NB stays essentially flat around 82-83% regardless of split ratio. This is a nice illustration of the two algorithms’ different appetites for data: KNN, having no explicit parameters to fit, benefits directly

from a denser neighborhood to search, whereas Gaussian NB’s per-class mean and variance estimates stabilize with comparatively little data, and simply don’t have much more to gain from extra samples.

Learning Curve / Feature Importance: not generated for this experiment. Neither Naïve Bayes nor KNN expose a built-in feature-importance score the way tree-based models do; if feature-level insight were needed, the correlation heatmap in Section 4 and a permutation-importance analysis would be the more appropriate tools, and are left as a possible extension.

11 Discussion

Two results stood out most. First, among the three Naïve Bayes variants, **Bernoulli NB** Two main findings stood out from the experiments. Among the Naïve Bayes models, Bernoulli Naïve Bayes performed the best, followed closely by Multinomial Naïve Bayes, while Gaussian Naïve Bayes gave the lowest performance. This matches what was observed during the data analysis, where most features were sparse and not normally distributed, making Bernoulli and Multinomial better suited for the dataset.

After tuning its parameters, KNN achieved the best overall results, giving the highest accuracy, F1-score, and ROC-AUC. These results were also consistent during cross-validation, showing that the improvement was reliable and not due to chance.

However, KNN was much slower when making predictions because it compares every new email with the training data. In contrast, Naïve Bayes made predictions almost instantly. For a real-world spam filter that needs to process thousands or even millions of emails quickly, this speed difference can be more important than the small improvement in accuracy. So, while KNN performed better in this experiment, Naïve Bayes may still be the more practical choice in real applications. Another interesting observation was that Grid Search and Randomized Search produced very similar hyperparameter values, showing that the tuning process was stable. The Ball Tree algorithm was also faster than KD Tree without affecting accuracy. Finally, using Manhattan distance with distance-weighted neighbors consistently gave better results than the standard Euclidean distance with uniform weighting, suggesting that giving more importance to closer neighbors works better for this dataset.

12 Conclusion

This experiment built and rigorously compared four spam/ham classifiers – Gaussian, Multinomial, and Bernoulli Naïve Bayes, and a hyperparameter-tuned KNN – on the Spambase dataset. Bernoulli NB was the strongest Naïve Bayes variant, consistent with the dataset’s sparse, skewed feature distributions. After tuning with GridSearchCV ($k = 9$, manhattan distance, distance-weighted voting, selected over RandomizedSearchCV by a narrow margin), KNN edged out every Naïve Bayes variant on accuracy, F1, and ROC-AUC, and this advantage was confirmed by 5-fold cross-validation rather than resting on a single lucky split. At the same time, the measured training and prediction times matched the theoretical complexity analysis closely: Naïve Bayes remains the cheaper option at prediction time and would likely be preferred at a much larger scale, while KNN is the better choice here purely on predictive performance. Overall, the experiment

underlines that the "best" classifier depends on what is actually being optimized for – raw accuracy, or the ability to serve predictions fast at scale.

13 References

- [1] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase Data Set," UCI Machine Learning Repository. [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>
- [2] Scikit-learn Developers, "scikit-learn: Machine Learning in Python," Scikit-learn Documentation. [Online]. Available: <https://scikit-learn.org>